Week 9: E-Commerce Sales and Customer Analysis System

An online shopping company wants to examine its sales activities to gain insights into customer orders, product sales, and business revenue. The company needs to analyze its sales information to identify popular products, understand customer purchasing patterns, and measure overall business performance.

1. Perform SQL Aggregate Operations

FUNCTION

- COUNT()
- SUM()
- AVG()
- MIN()
- MAX()

2.Generate Total Sales Report

1. **Total number of orders placed.**

```
SQL> SELECT COUNT(*) AS NUMBER_OF_ORDERS_PLACED FROM ORDERS;

NUMBER_OF_ORDERS_PLACED
-----------------------
                      8
```

2. **Total revenue generated from sales.**

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE FROM ORDERS;

TOTAL_REVENUE
-------------
       267800
```

3. **Average order value.**

```
SQL> SELECT ROUND(AVG(TOTAL_AMOUNT),2) AS AVERAGE_ORDER_VALUE FROM ORDERS;

AVERAGE_ORDER_VALUE
-------------------
              33475
```

4. Highest and lowest order amount.

```
SQL> SELECT MAX(TOTAL_AMOUNT)AS HIGHEST_ORDERS, MIN(TOTAL_AMOUNT) AS LOWER_ORDER FROM ORDERS;

HIGHEST_ORDERS LOWER_ORDER
-------------- -----------
        135000        1500
```

5.Total sales for a specific period.

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES FROM ORDERS WHERE ORDER_DATE BETWEEN TO_DATE('01-11-2026','DD-MM-YYYY') AND TO_DATE('30-11-20
26','DD-MM-YYYY');

TOTAL_SALES
-----------
     161600
```

CUSTOMER PURCHASE ANALYSIS

1.Total number of orders placed by each customer.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS TOTAL_ORDERS FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOMER_ID = O.CUST
OMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY C.CUSTOMER_ID;

CUSTOMER_ID CUSTOMER_NAME                                          TOTAL_ORDERS
----------- -------------------------------------------------- ------------
       1001 Arun                                                             3
       1002 Priya                                                            2
       1003 Kumar                                                            1
       1004 Divya                                                            1
       1005 Rahul                                                            1
       1006 Rohan                                                            0

6 rows selected.
```

2.Total amount spent by each customer.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, NVL(SUM(O.TOTAL_AMOUNT),0) AS TOTAL_AMOUNT_SPENT FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOME
R_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY C.CUSTOMER_ID;

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
TOTAL_AMOUNT_SPENT
------------------
       1001 Arun
             227500

       1002 Priya
              26600

       1003 Kumar
               1500


CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
TOTAL_AMOUNT_SPENT
------------------
       1004 Divya
               4200

       1005 Rahul
               8000

       1006 Rohan
                  0


6 rows selected.
```

3.Average spending per customer.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, NVL(AVG(O.TOTAL_AMOUNT),0) AS AVERAGE_SPENDING FROM CUSTOMER
 C LEFT JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY
C.CUSTOMER_ID;

CUSTOMER_ID CUSTOMER_NAME                                          AVERAGE_SPENDING
----------- -------------------------------------------------- ----------------
       1001 Arun                                                     75833.3333
       1002 Priya                                                         13300
       1003 Kumar                                                          1500
       1004 Divya                                                          4200
       1005 Rahul                                                          8000
       1006 Rohan                                                             0

6 rows selected.
```

4.Customer with maximum purchase amount.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE_AMOUNT FROM CUSTOMER C
 JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME HAVING SUM(O.TOT
AL_AMOUNT) = (SELECT MAX(TOTAL_SPENT) FROM (SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENT FROM ORDERS GROUP BY
 CUSTOMER_ID));

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
TOTAL_PURCHASE_AMOUNT
---------------------
       1001 Arun
               227500
```

5.Customers with fewer purchases.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS TOTAL_PURCHASES FROM CUSTOMER C LEFT JO
IN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME HAVING COUNT(O.ORDE
R_ID) = (SELECT MIN(ORDER_COUNT) FROM (SELECT COUNT(ORDER_ID) AS ORDER_COUNT FROM ORDERS GROUP BY CUSTOM
ER_ID));

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_PURCHASES
----------- -------------------------------------------------- ---------------
       1003 Kumar                                                            1
       1004 Divya                                                            1
       1005 Rahul                                                            1
```

4.FIND TOP CUSTOMERS BASED ON PURCHASE AMOUNT.

1.Top 5 customers based on total spending.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM CUSTOMER C JOIN O
RDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY TOTAL_SPENDING
 DESC FETCH FIRST 5 ROWS ONLY;

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_SPENDING
----------- -------------------------------------------------- --------------
       1001 Arun                                                       227500
       1002 Priya                                                       26600
       1005 Rahul                                                        8000
       1004 Divya                                                        4200
       1003 Kumar                                                        1500
```

2.Customers with maximum number of orders.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS TOTAL_ORDERS FROM CUSTOMER C JOIN ORDER
S O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME HAVING COUNT(O.ORDER_ID) =
(SELECT MAX(ORDER_COUNT) FROM (SELECT COUNT(ORDER_ID) AS ORDER_COUNT FROM ORDERS GROUP BY CUSTOMER_ID));


CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_ORDERS
----------- -------------------------------------------------- ------------
       1001 Arun                                                          3
```

3.Frequent customers.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS TOTAL_ORDERS FROM CUSTOMER C JOIN ORDER
S O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME HAVING COUNT(O.ORDER_ID) >
1 ORDER BY TOTAL_ORDERS DESC;

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_ORDERS
----------- -------------------------------------------------- ------------
       1001 Arun                                                          3
       1002 Priya                                                         2
```

4.High-value customers.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM CUSTOMER C JOIN O
RDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME HAVING SUM(O.TOTAL_AMOU
NT) > (SELECT AVG(TOTAL_SPENT) FROM (SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENT FROM ORDERS GROUP BY CUSTOM
ER_ID)) ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_ID CUSTOMER_NAME                                          TOTAL_SPENDING
----------- ---------------------------------------------------- --------------
       1001 Arun                                                         227500
```

5.IDENTIFY BEST-SELLING PRODUCTS.

1.Product with maximum sales quantity.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD FROM PRODUCT P JOIN OR
DER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME HAVING SUM(OD.QUANT
ITY) = (SELECT MAX(TOTAL_QTY) FROM (SELECT SUM(QUANTITY) AS TOTAL_QTY FROM ORDER_DETAILS GROUP BY PRODUC
T_ID));

PRODUCT_ID PRODUCT_NAME         TOTAL_QUANTITY_SOLD
---------- -------------------- -------------------
P1003      TShirt                                 4
```

2.Products generating highest revenue.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE FROM PRODUCT P JO
IN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME ORDER BY TOTAL
_REVENUE DESC;

PRODUCT_ID PRODUCT_NAME         TOTAL_REVENUE
---------- -------------------- -------------
P1001      Laptops                    202500
P1002      Mobile                      50000
P1005      Keyboard                     8000
P1004      Shoes                        4200
P1003      TShirt                       3100
```

3.Least-selling products.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, NVL(SUM(OD.QUANTITY),0) AS TOTAL_QUANTITY_SOLD FROM PRODUCT P
LEFT JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME HAVING
NVL(SUM(OD.QUANTITY),0) = (SELECT MIN(TOTAL_QTY) FROM (SELECT NVL(SUM(QUANTITY),0) AS TOTAL_QTY FROM PRO
DUCT P2 LEFT JOIN ORDER_DETAILS OD2 ON P2.PRODUCT_ID = OD2.PRODUCT_ID GROUP BY P2.PRODUCT_ID));

PRODUCT_ID PRODUCT_NAME         TOTAL_QUANTITY_SOLD
---------- -------------------- -------------------
P1006      Mouse                                  0
```

4.Product requiring promotion.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, NVL(SUM(OD.QUANTITY),0) AS TOTAL_QUANTITY_SOLD FROM PRODUCT P
LEFT JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME ORDER B
Y TOTAL_QUANTITY_SOLD ASC FETCH FIRST 5 ROWS ONLY;

PRODUCT_ID PRODUCT_NAME         TOTAL_QUANTITY_SOLD
---------- -------------------- -------------------
P1006      Mouse                                  0
P1005      Keyboard                               1
P1002      Mobile                                 2
P1004      Shoes                                  2
P1001      Laptops                                3
```

6.PERFORM CATEGORY-WISE SALES ANALYSIS

1.Total sales generated by each category.

```
SQL> SELECT P.CATEGORY_ID, SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_SALES FROM PRODUCT P JOIN ORDER_DETAILS
OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.CATEGORY_ID ORDER BY TOTAL_SALES DESC;

CATEGOR TOTAL_SALES
------- -----------
Cat0001      260500
Cat0003        4200
Cat0002        3100
```

2.Number of products sold in each category.

```
SQL> SELECT P.CATEGORY_ID, COUNT(DISTINCT P.PRODUCT_ID) AS TOTAL_PRODUCTS_SOLD, SUM(OD.QUANTITY) AS TOTA
L_QUANTITY_SOLD FROM PRODUCT P JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.CATEGORY
_ID;

CATEGOR TOTAL_PRODUCTS_SOLD TOTAL_QUANTITY_SOLD
------- ------------------- -------------------
Cat0001                   3                   6
Cat0002                   1                   4
Cat0003                   1                   2
```

3.Highest revenue-generating category.

```
SQL> SELECT P.CATEGORY_ID, SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE FROM PRODUCT P JOIN ORDER_DETAIL
S OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.CATEGORY_ID HAVING SUM(OD.QUANTITY * OD.PRICE) = (SELECT
 MAX(CATEGORY_REVENUE) FROM (SELECT SUM(OD2.QUANTITY * OD2.PRICE) AS CATEGORY_REVENUE FROM PRODUCT P2 JO
IN ORDER_DETAILS OD2 ON P2.PRODUCT_ID = OD2.PRODUCT_ID GROUP BY P2.CATEGORY_ID));

CATEGOR TOTAL_REVENUE
------- -------------
Cat0001        260500
```

4.Average sales per category.

```
SQL> SELECT P.CATEGORY_ID, AVG(OD.QUANTITY * OD.PRICE) AS AVERAGE_SALES FROM PRODUCT P JOIN ORDER_DETAIL
S OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.CATEGORY_ID ORDER BY P.CATEGORY_ID;

CATEGOR AVERAGE_SALES
------- -------------
Cat0001         52100
Cat0002          1550
Cat0003          4200
```

7.GENERATE BUSINESS ANALYTICS REPORTS.

REPORT 1: SALES PERFORMANCE REPORT.

- Total sales
- Number of orders
- Average order value
- Highest order value

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES, COUNT(ORDER_ID) AS NUMBER_OF_ORDERS, AVG(TOTAL_AMOUNT) AS
AVERAGE_ORDER_VALUE, MAX(TOTAL_AMOUNT) AS HIGHEST_ORDER_VALUE FROM ORDERS;

TOTAL_SALES NUMBER_OF_ORDERS AVERAGE_ORDER_VALUE HIGHEST_ORDER_VALUE
----------- ---------------- -------------------- -------------------
     267800                8                33475              135000
```

REPORT 2: CUSTOMER ANALYTICS REPORT

- Customer name
- Number of orders
- Total spending
- Purchase frequency

```
SQL> SELECT C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS, NVL(SUM(O.TOTAL_AMOUNT),0) AS TOTAL_SPENDING, CASE WHEN COUNT(O.ORDER_
ID) = 0 THEN 'NO PURCHASES' WHEN COUNT(O.ORDER_ID) = 1 THEN 'OCCASIONAL' ELSE 'FREQUENT' END AS PURCHASE_FREQUENCY FROM CUSTOMER C LEFT JO
IN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_NAME                                      NUMBER_OF_ORDERS
-------------------------------------------------- ----------------
TOTAL_SPENDING PURCHASE_FRE
-------------- ------------
Arun                                                              3
        227500 FREQUENT

Priya                                                             2
         26600 FREQUENT

Rahul                                                             1
          8000 OCCASIONAL


CUSTOMER_NAME                                      NUMBER_OF_ORDERS
-------------------------------------------------- ----------------
TOTAL_SPENDING PURCHASE_FRE
-------------- ------------
Divya                                                             1
          4200 OCCASIONAL

Kumar                                                             1
          1500 OCCASIONAL

Rohan                                                             0
             0 NO PURCHASES


6 rows selected.
```

REPORT3: PRODUCT PERFORMANCE REPORT

- Product name
- Quantity sold
- Revenue generated

```
SQL> SELECT P.PRODUCT_NAME, NVL(SUM(OD.QUANTITY),0) AS QUANTITY_SOLD, NVL(SUM(OD.QUANTITY * OD.PRICE),0)
 AS REVENUE_GENERATED FROM PRODUCT P LEFT JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY
 P.PRODUCT_ID, P.PRODUCT_NAME ORDER BY REVENUE_GENERATED DESC;

PRODUCT_NAME         QUANTITY_SOLD REVENUE_GENERATED
-------------------- ------------- -----------------
Laptops                          3            202500
Mobile                           2             50000
Keyboard                         1              8000
Shoes                            2              4200
TShirt                           4              3100
Mouse                            0                 0

6 rows selected.
```

REPORT 4: CATEGORY ANALYSIS REPORT

- Category name
- Total product sold
- Total revenue

```
SQL> SELECT P.CATEGORY_ID AS CATEGORY_NAME, COUNT(DISTINCT P.PRODUCT_ID) AS TOTAL_PRODUCTS_SOLD, NVL(SUM
(OD.QUANTITY * OD.PRICE),0) AS TOTAL_REVENUE FROM PRODUCT P LEFT JOIN ORDER_DETAILS OD ON P.PRODUCT_ID =
 OD.PRODUCT_ID GROUP BY P.CATEGORY_ID ORDER BY TOTAL_REVENUE DESC;

CATEGOR TOTAL_PRODUCTS_SOLD TOTAL_REVENUE
------- ------------------- -------------
Cat0001                   4        260500
Cat0003                   1          4200
Cat0002                   1          3100
```